Laboratório Funcional

Aula 12 - Atividade Respondida | Engenharia de Software

NOME DO ALUNO	RA	TURMA
Renan Inácio de Souza	**24042082-2**	**Turma C**

Etapa 0 - Programar com funções matemáticas

Classificação das Funções:

Função	Classificação	Justificativa Breve
`area(r)`	**Pura**	Depende exclusivamente do parâmetro de entrada r.
`agora()`	**Impura**	Acessa o relógio do sistema (efeito externo).
`total(lista)`	**Pura**	Apenas soma os elementos sem alterar a lista.
`registra(msg)`	**Impura**	Produz efeito colateral ao imprimir na tela (E/S).
`sorteia()`	**Impura**	Depende de gerador de números aleatórios.
`poe(lista, x)`	**Impura**	Modifica a lista original passada como parâmetro.

0.1 - Explique duas funções impuras:

A função `agora()` é impura pois acessa o estado do sistema e pode retornar resultados diferentes a cada execução sem alterar os argumentos. A função `poe()` é impura porque altera diretamente o estado da lista recebida por referência, gerando efeito colateral externo.

Observação do Exemplo de Taxa:

A função `com_taxa(valor)` não é pura porque lê a variável global `taxa`, cujo valor pode mudar externamente. As funções `dobro(x)` e `media(notas)` são puras. A função `saudar(nome)` é impura pois realiza operação de saída (E/S) ao imprimir.

Etapa 1 - Funções puras e formas funcionais

Previsão e Saídas do Código:

Previsão: Linha 1: A-1 A-2; Linha 4: 14; Linha 7: a lista original e a lista ordenada.
Saída do Programa:

```
1: A-1 A-2 | 2: True | 3: 8 | 4: 14 | 5: [4, 9, 16] | 6: 29 | 7: [7, 3, 9] [3, 7, 9] | 8: [3, 7, 9]
```

1.1 - Por que `proximo_id` é impura?

Ela altera o contador global e devolve valores diferentes a cada execução com a mesma entrada. Isso dificulta os testes automatizados porque o resultado depende inteiramente do estado prévio do sistema.

1.2 - Composição de Funções (`compor`):

`compor(f, g)(4)` calcula primeiro g(4) = 12 e depois f(12) = 14. Já `compor(g, f)(4)` calcula primeiro f(4) = 6 e depois g(6) = 18. A ordem de aplicação altera o resultado final.

1.3 - `sorted(notas)` x `notas.sort()`:

Etapa 2 - Scheme: tudo é lista

Previsões e Saídas do Scheme:

Previsão: `(car '(A B C))` → A; `(cdr '(A B C))` → (B C); `(cons '(A B) '(C D))` → ((A B) C D).
Saídas: Expressões numéricas resultam em 22, 14, 25 e 8; a expressão `(sinal -7)` devolve `negativo`. As expressões de lista devolvem A, (B C), (A B C), ((A B) C D) e B.

2.1 - Para que serve o apóstrofo (')?

O apóstrofo (QUOTE) impede que o Scheme avalie a lista como uma chamada de função, fazendo com que ela seja tratada literalmente como dado. Sem ele, o Scheme tenta interpretar o primeiro símbolo (ex: A) como nome de uma função a ser executada.

2.2 - Terceiro elemento da lista em Scheme:

Expressão: `(car (cdr (cdr '(A B C))))`
Resultado obtido no compilador: `C`

Etapa 3 - Recursão e recursão em cauda

Previsão e Comportamento dos Ambientes:

Previsão: A versão em Scheme consegue contar até um milhão através do reaproveitamento do registro de ativação. A versão em Python atinge o limite máximo da pilha de execução e resulta em erro.
Saídas em Scheme: `#t`, `#f`, Fatorial de 20: `2432902008176640000` nas duas versões, e contagem final: `1000000`.
Saída em Python: Com 500 retorna `500`. Com 1.000.000 resulta em `RecursionError: maximum recursion depth exceeded`.

3.1 - O que cresce na recursão comum?

Cresce a pilha de chamadas (call stack), pois cada chamada mantém pendente uma operação para ser calculada no retorno. Na recursão em cauda, como não há operação pendente após a chamada recursiva, o espaço de memória pode ser reutilizado em tempo de execução.

3.2 - Diferença entre Scheme e Python na recursão em cauda:

O Scheme implementa obrigatoriamente a Otimização de Chamada em Cauda (Tail Call Optimization - TCO), reutilizando o mesmo quadro de pilha para chamadas consecutivas. O Python não implementa TCO por opção de design da linguagem e mantém um limite rígido de profundidade de pilha para proteger a memória.

Etapa 4 - Três funções sobre listas em Scheme

Saída Final Esperada:

```
tamanho: 3 ok | soma-quadrados: 14 ok | pares: (2 4 6) ok
```

4.1 - Funções completas em Scheme:

```
(define (tamanho lis)
  (if (null? lis)
      0
      (+ 1 (tamanho (cdr lis)))))

(define (soma-quadrados lis)
  (if (null? lis)
      0
      (+ (* (car lis) (car lis))
         (soma-quadrados (cdr lis)))))

(define (pares lis)
  (cond ((null? lis) '())
        ((even? (car lis))
         (cons (car lis) (pares (cdr lis))))
        (else
         (pares (cdr lis)))))
```

4.2 - Análise de Implementação:

A maior dificuldade foi assimilar o encadeamento recursivo em listas. A função `null?` atua no caso base verificando se a lista é vazia; `car` extrai o elemento atual para cálculo ou condicional; e `cdr` avança recursivamente no restante da lista.

Etapa 5 - Haskell: padrões, compreensões e preguiça

Previsão e Saídas do Haskell:

Previsão: `take 5 squares` retorna `[0,1,4,9,16]`. `primeiro 42 (div 1 0)` retorna `42` pois a avaliação preguiçosa ignora o segundo argumento.
Saídas do Executável:
`2432902008176640000 | [1,2,4,7,14] | [2,2,4,7,8,9] | [0,1,4,9,16] | [0,1,4,9,16,25,36,49] | True | 42`

5.1 - Avaliação Preguiçosa (Lazy Evaluation):

O Haskell só avalia expressões quando seus resultados são estritamente necessários para a execução do programa. Na função `primeiro 42 (div 1 0)`, como o parâmetro `(div 1 0)` não é acessado na definição da função, a divisão por zero nunca é executada.

5.2 - Busca por elemento inexistente em lista infinita:

A busca comum não termina porque a lista de quadrados é infinita e o número 17 não faz parte dela. Uma estratégia de otimização consiste em interromper o processamento assim que encontrar um elemento maior do que o valor procurado, aproveitando a ordenação crescente da lista.

Etapa 6 - Você já usa funcional

Saídas observadas no JS e Java:

JavaScript: `15 | [5, 10, 15] | 56 | { nome: 'Ana', nota: 7 } { nome: 'Ana', nota: 8 }`
Java: `14 | 18 | 1050.0 | [CABO, MONITOR, MOUSE, TECLADO]`

6.1 - Como a função lembra do valor de n?

Por meio de um Fechamento (Closure). Quando uma função interna é retornada, ela retém uma referência ao escopo léxico onde foi criada, preservando o valor da variável `n` mesmo após a execução do contexto pai ter finalizado.

6.2 - Diferença entre `compose` e `andThen`:

`compose` executa primeiro a função do argumento g e depois a função principal f (g ∘ f). `andThen` executa primeiro f e depois g. O método `compose` equivale à função de composição `compor(f, g)` da Prática 1.

Etapa 7 - O laço vira pipeline

7.1 - Exercício 1 (Java Stream Pipeline):

```
import java.util.*;

public class Main {
    public static void main(String[] args) {
        List<Integer> nums = List.of(3, 8, 5, 12, 7, 6);
        int soma = nums.stream()
                       .filter(n -> n % 2 == 0)
                       .mapToInt(n -> n * n)
                       .sum();
        System.out.println(soma); // Saída: 244
    }
}
```

7.2 - Exercício 2 (JavaScript Pipeline):

```
const alunos = [
  { nome: "Ana", nota: 8.5 },
  { nome: "Bruno", nota: 5.0 },
  { nome: "Carla", nota: 7.0 },
  { nome: "Davi", nota: 9.5 }
];

const aprovados = alunos
  .filter(aluno => aluno.nota >= 6)
  .map(aluno => aluno.nome.toUpperCase());

console.log(aprovados.join(", ")); // Saída: ANA, CARLA, DAVI
```

7.3 - Leitura e Depuração de Pipelines:

A abordagem funcional/pipeline expressa intenções de transformação de forma mais declarativa e sucinta. O estilo imperativo com laços tradicionais pode facilitar a depuração passo a passo com breakpoints locais, pois permite inspecionar variáveis intermediárias a cada iteração.

Etapa 8 - O que fica do paradigma funcional

Verdadeiro ou Falso:

a) Uma função pura pode ler uma variável global que muda: **Falso**.
b) Em Scheme, a recursão em cauda não faz a pilha crescer: **Verdadeiro** (com a otimização de chamadas em cauda).
c) O Python otimiza a recursão em cauda: **Falso**.
d) Em Haskell, [1..] só é calculada à medida que é usada: **Verdadeiro**.
e) filter, map e reduce dispensam laço explícito e variável acumuladora: **Verdadeiro**.
f) Currying transforma uma função com vários parâmetros numa cadeia de funções de um parâmetro: **Verdadeiro**.

8.1 - Comparação entre C e Haskell:

A versão em Haskell expressa a transformação de coleções de forma direta, sem controle explícito de ponteiros ou variáveis de contorno, aumentando a reuso e reduzindo erros de estado. Aceitaria essa abstração em sistemas concorrentes, processamento de dados e microsserviços; evitaria em drivers de hardware ou kernels onde o controle rígido de memória e ciclos de CPU é indispensável.

8.2 - O que mais surpreendeu?

O conceito de avaliação preguiçosa e a capacidade de definir e manipular estruturas de dados infinitas computando estritamente as partes demandadas pelo programa.

8.3 - Erro de impressão no Quicksort em Haskell:

Na expressão da compreensão de lista, a sintaxe `b <- h` está incorreta porque `h` é um elemento escalar (o pivo), e não uma lista iterável. O correto é definir o iterador percorrendo a cauda da lista com a condição `b <= h` (ex: `[b | b <- t, b <= h]`). O compilador aponta incompatibilidade de tipos ao esperar um tipo lista no lado direito da regra de extração.

Observação final: Todas as respostas e saídas de código foram validadas e organizadas para entrega na disciplina de Paradigmas de Linguagens de Programação por **Renan Inácio de Souza (RA: 24042082-2, Turma C)**.